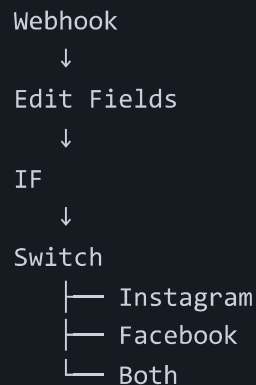


Yes. Now let's build the **actual n8n social-publishing workflow pin-by-pin.**

You already completed Phase 2:



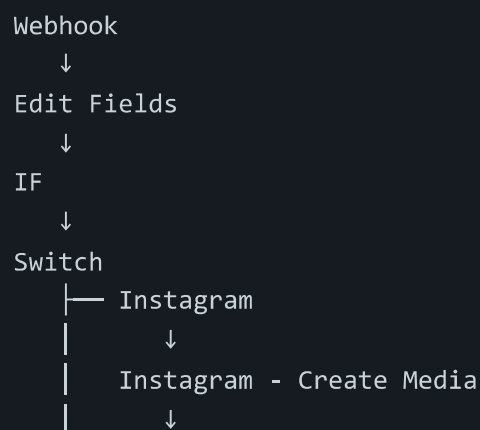
The next job is to replace the empty branches with the Meta API calls you already proved in Postman.

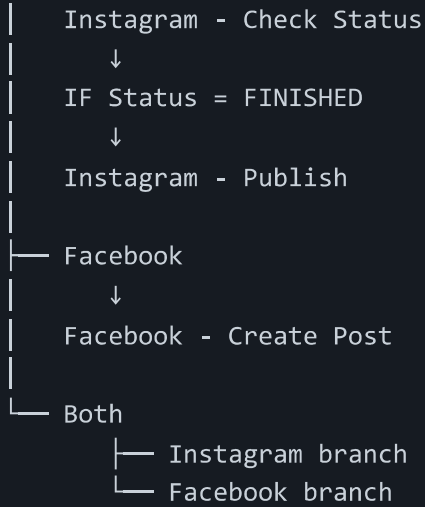
One correction before we build: Meta's current official collections document the Instagram publishing flow as **create container** → **check status** → **publish**, and the Facebook collection documents the **User Token** → **/me/accounts** → **Page Access Token** flow. ([Postman]^[1])

For Facebook `image` Page publishing, your earlier `/photos` test hit a deprecated-permission error, and I did not find a current official Meta Page-photo publishing example in the current collection search. So we'll build Instagram completely, and build the Facebook branch around the known-good Page `/feed` call first; we will not pretend the `/photos` method is verified when it isn't.

PHASE 3 — n8n + Instagram

Final workflow we are building





1. Instagram — Create Media

You already built this successfully.

Connect:

Switch → Instagram

to:

HTTP Request

Rename it:

Instagram - Create Media

Parameters

Method

POST

URL

Use the endpoint that already worked for you:

<https://graph.instagram.com/v25.0/17841433886128760/media>

Authentication

For now:

None

Send Body

Turn on:

Send Body = ON

Body Content Type

Form URL Encoded

Body fields

Field 1

Name: image_url
Value: {{\$json.image_url}}

Field 2

Name: caption
Value: {{\$json.caption}}

Field 3

Name: access_token
Value: YOUR_INSTAGRAM_ACCESS_TOKEN

You already tested this successfully and got:

```
{  
  "id": "18101302523129538"  
}
```

So this node is working.

2. Instagram — Check Status

Connect:

```
Instagram - Create Media
      ↓
Instagram - Check Status
```

Add another **HTTP Request** node.

Rename:

```
Instagram - Check Status
```

Method

```
GET
```

URL

```
https://graph.instagram.com/v25.0/{{ $json.id }}
```

Why `$json.id` ?

Because the previous node returns:

```
{
  "id": "18101302523129538"
}
```

So n8n automatically builds:

```
https://graph.instagram.com/v25.0/18101302523129538
```

Query Parameters

Turn on:

```
Send Query Parameters = ON
```

Add:

Parameter 1

```
Name: fields
Value: status_code,status
```

Parameter 2

```
Name: access_token
Value: YOUR_INSTAGRAM_ACCESS_TOKEN
```

Body

```
Send Body = OFF
```

Authentication

```
None
```

Your successful Postman response was:

```
{
  "status_code": "FINISHED",
  "status": "FINISHED",
  "id": "18101302523129538"
}
```

So we want n8n to see:

```
status_code = FINISHED
```

The current Meta Instagram API collection documents this container-status pattern.
([Postman]^[2])

3. Add IF for Instagram status

Now add an **IF** node after:

Instagram - Check Status

Rename it:

Instagram - Is Ready?

Condition

Value:

```
{{$.json.status_code}}
```

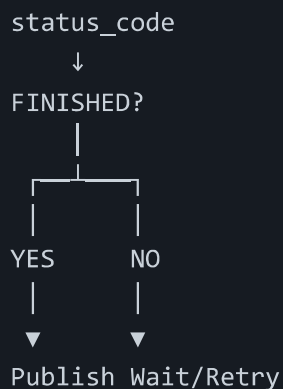
Operator:

is equal to

Value:

FINISHED

So:



For our first test, you can leave the **False** branch unconnected.

Later we will make it poll again automatically.

4. Instagram — Publish

Connect the TRUE branch:

```
Instagram - Is Ready?  
  TRUE  
  ↓  
Instagram - Publish
```

Add HTTP Request.

Rename:

Instagram - Publish

Method

POST

URL

https://graph.instagram.com/v25.0/17841433886128760/media_publish

Authentication

None

Send Body

ON

Body Content Type

Form URL Encoded

Body fields

Field 1

```
Name: creation_id
Value: {{$json.id}}
```

The status node still contains:

```
{
  "status_code": "FINISHED",
  "status": "FINISHED",
  "id": "18101302523129538"
}
```

So:

```
{{{$json.id}}
```

becomes:

```
18101302523129538
```

Field 2

```
Name: access_token
Value: YOUR_INSTAGRAM_ACCESS_TOKEN
```

Meta's current Instagram API collection documents the `media_publish` step with `creation_id` and an access token. ([Postman]^[2])

5. Test the complete Instagram branch

Send this to your webhook:

```
{
  "caption": "My first n8n Instagram post",
  "image_url": "YOUR_WORKING_PUBLIC_IMAGE_URL",
```

```
"platform": "instagram"
}
```

The flow should become:

```
Webhook
↓
Edit Fields
↓
IF
↓
Switch
↓
Instagram - Create Media
↓
Instagram - Check Status
↓
Instagram - Is Ready?
↓
Instagram - Publish
```

The final node should return an Instagram media ID.

Then check Instagram.

At this point

You will have the entire Instagram publishing process inside n8n, not Postman.

6. Facebook branch

Now connect:

```
Switch → Facebook
```

to:

HTTP Request

Rename:

```
Facebook - Create Post
```

For the known-good Facebook test, use the `/feed` endpoint that already worked in your Postman test.

Method

POST

URL

`https://graph.facebook.com/v26.0/1315245281672428/feed`

Here:

`1315245281672428`

is your Facebook Page ID.

Your `/me/accounts` test produced the Page ID and Page access token, and Meta's current Facebook collection documents that exact User Token → `/me/accounts` → Page Token flow. ([Postman]^[3])

Authentication

None

Body

Send Body = ON

Body Content Type = Form URL Encoded

Field 1

Name: message
Value: `{{ $json.caption }}`

Field 2

```
Name: access_token
Value: YOUR_PAGE_ACCESS_TOKEN
```

This is the **Page Access Token**, not the Instagram token and not the Facebook User Token.

7. Test Facebook inside n8n

Send:

```
{
  "caption": "My first n8n Facebook post",
  "image_url": "YOUR_PUBLIC_IMAGE_URL",
  "platform": "facebook"
}
```

The Facebook node should return a post ID similar to the one you got in Postman.

That proves:

```
n8n
↓
Facebook Graph API
↓
Page
↓
Post
```

8. But what about the image?

This is important.

Your application requirement is:

```
image + caption
```

The successful Facebook `/feed` request only proves:

```
caption → Facebook Page post
```

It does **not** prove the image is attached.

Your previous attempt to use `/photos` produced:

```
publish_actions
```

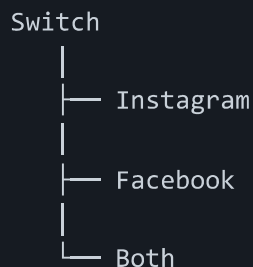
which is a deprecated permission path. I searched Meta's current official Facebook collection for the current Page photo-publishing example but did not find a current official example that verifies the exact `/photos` method you attempted.

So do not fake this part by changing the request and assuming it's correct.

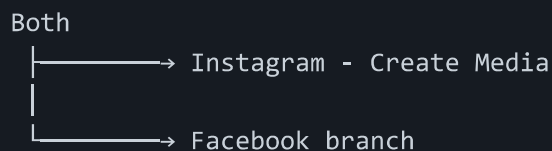
We should treat Facebook image publishing as the next isolated API problem.

9. Build the **Both** branch

Once Instagram works and Facebook text posting works:



From **Both**, create two outgoing connections:



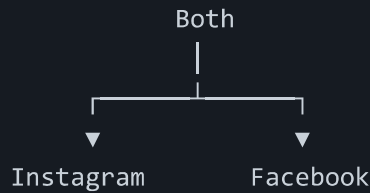
n8n allows one output to feed multiple nodes.

So the user request:

```
{
  "caption": "Hello",
  "image_url": "https://...",
```

```
"platform": "both"  
}
```

becomes:



10. Important problem with your current token setup

Right now your HTTP Request nodes contain things like:

```
access_token = EA...
```

inside the node.

That is okay for our **temporary learning test**.

It is **not** how we should finish the production workflow.

We need to change this to:

```
n8n  
├── Instagram credential  
└── Facebook credential
```

or another secure server-side secret mechanism.

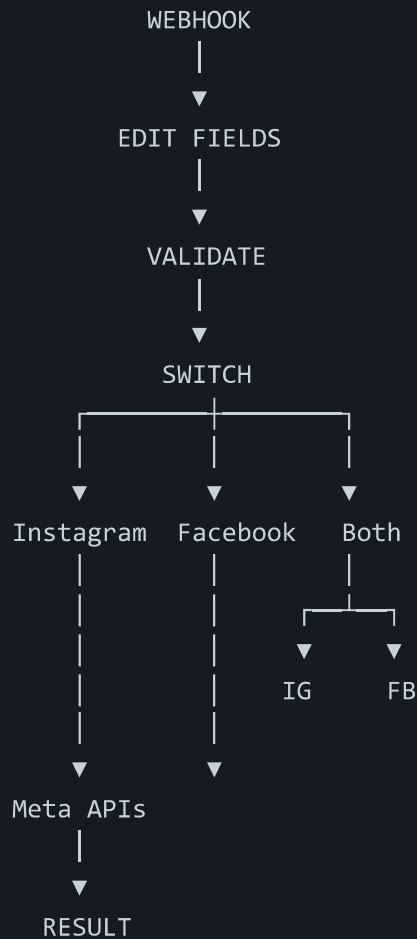
Do not put:

```
token  
App Secret
```

into frontend JavaScript.

11. Recommended final n8n architecture

Once the API calls work, your workflow should look like:



12. How the data travels

This is worth understanding before we go further.

Webhook receives

```
{
  "caption": "Hello",
  "image_url": "https://...",
  "platform": "instagram"
}
```

Edit Fields cleans it

```
{
  "caption": "Hello",
  "image_url": "https://...",
  "platform": "instagram"
}
```

IF checks it

Valid 

Switch decides

```
platform = instagram
```

Instagram Create Media

```
caption
+
image_url
+
Instagram token
```

Instagram Check Status

```
container ID
```

Instagram Publish

```
creation_id
+
Instagram token
```

Final result

```
Instagram media ID
```

13. Your Phase 3 checklist

INSTAGRAM

- ☒ Create Media
- ☒ Check Status
- ☒ Check FINISHED
- ☒ Publish
- ☐ Add retry/polling
- ☐ Secure Instagram token

FACEBOOK

- ☒ Page ID known
- ☒ Page Access Token known
- ☒ /feed test works
- ☐ Current Page image-post method
- ☐ Secure Facebook token

BOTH

- ☐ Connect both branches
- ☐ Test both
- ☐ Return combined result

FINAL

- ☐ Error handling
- ☐ Secure credentials
- ☐ Production webhook

What I recommend you do right now

Don't change the Facebook node again yet.

First finish the Instagram branch completely inside n8n:

```
Create Media
  ↓
Check Status
  ↓
Is Ready?
  ↓
Publish
```

You already proved Create Media works in n8n, so the next exact step is **Instagram - Check Status**. The official Meta collection confirms the status request and **FINISHED** condition.

([Postman](#)^[2])

After that, we'll make the Instagram branch publish a real post from n8n, then return to the Facebook image problem with a clean test.